

Case Study: AI Loan Risk and Eligibility Classifier

Rishav Raj
PRN - 2414110269

Summary

I looked at a small Streamlit app that trains two classifiers on made-up loan data and lets you try an applicant against either one. The Decision Tree gets 100% accuracy and Gaussian Naive Bayes gets about 87.5%. My first reaction was that the tree must simply be the better model. After reading the code, I think the gap says more about how the data was made than about the algorithms.

Whether a loan is approved comes from a fixed rule: credit score of 650 or more, and an income-to-loan ratio of 1.5 or more. A Decision Tree can find that with two splits, and it does. Naive Bayes has a harder time with an AND condition, so it gets things wrong near the edges. As a teaching demo for comparing models, the app does its job well. As evidence of how credit scoring works in real life, it doesn't tell you much.

Every figure here comes from running the app's own data generation and training code, with seed 42 and an 80/20 stratified split.

Background

When a bank decides on a loan, it is really guessing whether the person will pay it back. For a machine-learning exercise that becomes a yes/no problem: given a few details about an applicant, predict Approved (1) or Rejected (0).

Real lending data is private and tightly regulated, so people building demos or learning the basics usually make up their own. This project does that. It creates between 10,000 and 100,000 fictional applicants, labels them with a simple rule anyone can read, and trains two common classifiers to see how well each one works the rule out.

What I wanted to know was how a model that learns rules (the Decision Tree) stacks up against one that works with probabilities (Gaussian Naive Bayes) on the same task, and whether someone with no technical background could poke at the predictions and get something out of it.

What the app set out to do

The goals were fairly modest. The app should create a reproducible synthetic dataset in a size the user picks, train and compare the two models, and report accuracy, precision, recall, F1 and confusion matrices. It should also let someone type in an income, a credit score and a loan amount and see a prediction with an approval probability. And it should say plainly, in the interface, that the data is synthetic and the results are for learning only.

A few things were left out on purpose: real bank data, hyperparameter tuning, cross-validation, fairness testing and deployment.

How it's built

Everything lives in one Python script, which Streamlit re-runs from the top each time the user clicks something. I count ten labelled sections in it.

Layer	Tool	What it does
Interface	Streamlit	Layout, sidebar slider, tabs, metrics, buttons, CSV download
Data	NumPy, Pandas	Random data generation and tabular handling
Modelling	scikit-learn	Train/test split, the two classifiers, metrics
Plots	Matplotlib	Confusion-matrix heatmaps

Going down the page, you first meet the configuration and styling, then the sidebar where you choose a dataset size, then data generation and a row of overview metrics. Next comes a collapsible data inspector with a preview, statistics and the class distribution. After that the app trains the models and calculates metrics, and the main part of the page puts a performance dashboard next to the applicant predictor. A footer closes it out.

To keep it quick, the dataset is cached with `st.cache_data` and the trained models with `st.cache_resource`. Both caches depend on the record count.

The data

Each record is drawn from independent uniform distributions using a seeded generator (`default_rng(42)`), so you get the same data every run.

Field	Range	Used by models?
Income (₹)	20,000 to 200,000	Indirectly, through the ratio
CreditScore	300 to 850	Yes
LoanAmount (₹)	5,000 to 80,000	Indirectly, through the ratio
IncomeLoanRatio	Income ÷ LoanAmount	Yes
Approved	1 if CreditScore ≥ 650 and ratio ≥ 1.5, otherwise 0	Target

The rule approves roughly 27.5% of applicants at all four dataset sizes. At the default 20,000 records the figure is 27.6%, which is 5,514 approved and 14,486 rejected. That's a moderate imbalance, and it's the reason the split is stratified: train and test sets keep the same share of approvals.

The models only see two features, CreditScore and IncomeLoanRatio. I'd say the ratio matters more of the two, since it squeezes affordability into one number.

The two models

Both are trained on 80% of the data and tested on the other 20%. At the default size that means 16,000 records for training and 4,000 for testing, split with `random_state=42` and stratified on the approval label.

	Decision Tree	Gaussian Naive Bayes
Idea	Asks a series of threshold questions	Models each feature as a bell curve per class, then applies Bayes' rule
Setting	<code>max_depth=3</code>	Defaults
Assumption	Nothing about the distributions	Features are independent within each class and roughly normal
Output	Class, plus a probability from leaf purity	Class, plus a probability from the fitted curves

	Decision Tree	Gaussian Naive Bayes
Fit for this task	Natural, since the label is a pair of thresholds	Poor, since the label is an AND of two thresholds on uniform data

The tree that comes out at 20,000 records has just three questions. If the credit score is 649.5 or lower, reject. If it passes that and the ratio is 1.50 or lower, reject. Otherwise approve. That is the original rule, found exactly.

Results

The Decision Tree is almost perfect at every dataset size. Naive Bayes hovers around 87.5%. These are the numbers at the default 20,000 records, which gives 4,000 test cases.

Metric	Decision Tree	Gaussian Naive Bayes
Accuracy	100.00%	87.52%
Precision	100.00%	74.63%
Recall	100.00%	82.96%
F1-score	100.00%	78.57%

Confusion matrices (rows are actual, columns are predicted):

20,000 records	Predicted Rejected	Predicted Approved
Tree: actually Rejected	2,897	0
Tree: actually Approved	0	1,103
Naive Bayes: actually Rejected	2,586	311
Naive Bayes: actually Approved	188	915

Accuracy at each dataset size:

Records	Decision Tree	Gaussian Naive Bayes
10,000	100.00%	87.40%
20,000	100.00%	87.52%
50,000	100.00%	88.22%
100,000	99.99% (1 error in 20,000)	88.11%

Naive Bayes goes wrong in two ways. It approves 311 applicants who fail the rule (false positives) and turns down 188 who meet it (false negatives). For a lender the false positives are the expensive ones. With precision at 74.6%, about one approval in four is a mistake. Giving it more data doesn't help, because the limit comes from what the model assumes and has little to do with sample size.

The applicant predictor

The panel on the right is where a non-technical person can play with the models. You enter annual income, drag a slider for credit score, type a loan amount, and choose either Decision Tree or Gaussian Naive Bayes.

Click Evaluate Credit Risk and the app works out the income-to-loan ratio, then builds a one-row table using the same two features as in training. You then see the ratio, the rule behind the data, a green Approved or red Rejected verdict, the model's approval probability and a progress bar.

Below that, the app compares the model's answer with the original rule, says whether they agree, and adds a short note about why they can differ near the thresholds. A caption reminds you it's a demo on synthetic data.

The agreement check is my favourite part, because it shows the model's mistakes openly. I also like that the code finds the approval class with `classes_.index(1)` instead of assuming which column it sits in.

What works and what doesn't

The main problem is circularity. The label is a fixed function of the same features the models are given, so a score of 100% comes with the setup and isn't something the tree earned. No real dataset has a clean, noise-free approval rule built on two thresholds.

What I liked

- The structure is clean and easy to follow, with numbered sections and sensible caching.
- Results can be reproduced, thanks to the fixed seed and stratified split.
- It shows four metrics and confusion matrices, so you aren't left with accuracy alone.
- It's upfront about its limits, with repeated warnings that the data is synthetic and shouldn't drive real decisions.

Problems I found in the code

Issue	Where	Effect	Fix
Label is the rule itself	load_data	Tree scores 100%, so the comparison favours it	Add noise, more variables and a probabilistic label
"Features" metric shows 4	Overview metrics	Models only use 2 (CreditScore, IncomeLoanRatio)	Show <code>len(FEATURES)</code> or explain the difference
Predictor accepts inputs outside the training range	Income up to ₹500,000 and loan up to ₹150,000, versus ₹200,000 and ₹80,000 in training	Predictions extrapolate beyond anything the model saw	Match the input limits to the data, or show a warning
Naive Bayes independence assumption	GaussianNB	About 87.5% accuracy, and probabilities are unreliable near the thresholds	Present the probability as uncalibrated, or calibrate it
Unused variables	X, y, test_size	Dead code, and the split is hard-coded again inside <code>train_models</code>	Remove them or pass <code>test_size</code> through
Single split, no cross-validation	train_models	One number per metric, with no sense of variation	Use k-fold cross-validation
Shallow tree is hard-coded	max_depth=3	Only works because the rule needs two splits	Tune the depth, or show a depth-versus-accuracy curve
Inconsistent naming and scale	Page title says "AI Loan Risk & Approval Engine" while the heading says "...Eligibility Classifier"; score slider stops at 850	Minor confusion, and CIBIL scores run to 900	Align the names and the range

Two things deserve a mention. First, Naive Bayes isn't a bad model in general. It just doesn't suit a task defined by sharp thresholds on evenly spread inputs. Second, the app's own warning that probabilities are "not a guarantee" is fair, because neither model's probability says anything real

about default risk.

Ethical and responsible AI considerations

In a classroom the app is harmless. The same design inside a real lender would worry me for several reasons.

- **Bias.** Income and credit history can quietly stand in for gender, caste, region or age. Synthetic data hides the problem because it has no demographic columns to check.
- **Explainability.** Someone who is turned down is entitled to know why. A Decision Tree is easy to explain, whereas Naive Bayes is much harder to defend to a customer.
- **Cost of errors.** A false approval can end in a defaulted loan, and a false rejection keeps credit from someone who deserved it. The app weighs both the same, and a lender wouldn't.
- **Human oversight.** A person should be able to review automated credit decisions.
- **Compliance.** Real deployments in India have to follow RBI guidance and data-protection law, and a demo doesn't deal with either.

The disclaimers already in the interface are a good start, and I'd keep them.

What I'd do next

If I were taking this further, I'd start with the data. Adding noise, correlated variables (income and credit score usually move together) and a probabilistic approval label would mean no model could reach 100%. More features would help too: employment type, existing debt, loan tenure and past defaults would turn a threshold exercise into a real modelling problem.

After that, I'd compare more models, such as logistic regression, random forest and gradient boosting, using cross-validation and tuned hyperparameters. Class weights or a custom threshold would handle the imbalance, and a cost-sensitive metric should sit next to F1. I'd also calibrate the probabilities and show a reliability curve, and add explanations through the tree diagram, feature importances or SHAP values for each applicant.

On the fairness side, I'd put protected attributes into the synthetic data purely to measure error gaps between groups. Finally, the code needs tidying: remove the unused variables, fix the feature count, match the input ranges to the training data, and split the script into data, model and UI modules.

Conclusion

As an introduction to classification, evaluation metrics and model comparison, the project works well and is pleasant to use. The lesson I took from it is one it teaches by accident: a model that scores 100% on data built from a simple rule has learned that rule and little else.

- A Decision Tree matches threshold-based labels exactly (100%). Gaussian Naive Bayes reaches about 87.5%, because its independence and normality assumptions don't suit the data.
- Dataset size barely changed anything, which tells me the Naive Bayes gap comes from the model itself.
- A realistic evaluation needs noisy, richer data, cross-validation and metrics that reflect cost.
- Any real lending use would need fairness checks, explainability and regulatory review, none of which a synthetic demo can offer.